automatically creates it for you. A sharded actor is not an `ActorRef`; it is an `EntityRef`, but the API is similar.

In the preceding snippet, you can see that the SR is built using the actor's key—`Container.TypeKey`—and a function that sets the `containerId` from the `EntityContext`'s `entityId`. The similarity between the `EntityContext` and the `ActorContext`—from an `ActorRef`—is practically nonexistent. `EntityContext` contains little more than the `EntityTypeKey` and the `EntityId: String`. Among other things, this means an `EntityContext` has no `watch` as you would find in an `ActorContext` and no methods for logging, spawning, or scheduling, to name a few.

9.1.3 The simplicity of the sharded entities

If the `EntityContext` had `watch` as a method, the entity abstraction could be monitored to be notified on termination, but that contradicts the idea of an entity. The system handles the lifecycle and rebalancing.

The essence of an entity is that its creation and maintenance are overseen by the sharding module, not by you. This does not mean it is impossible to watch the entity; you can still get a reference to its `ActorRef` from `self` and interact with it as you would with a normal actor. However, doing so can have unexpected consequences. For example, if you send messages directly to the `ActorRef`, the passivation mechanism—which adds or removes entities from memory depending on their activity—will not register those messages as activity (more about passivation later in this chapter).

The following listing has the same code as in the previous snippet, but in the context of a test with a probe that checks whether the container has the expected cargo after cargo is added.

Listing 9.2 Creating cluster shading and an SR and sending messages to an entity

```
import akka.cluster.sharding.typed.scaladsl.{                    ❶
  ClusterSharding,
  Entity,
  EntityRef,
  EntityTypeKey
}
import akka.cluster.sharding.typed.ShardingEnvelope              ❶

class ContainerSpec
    extends ScalaTestWithActorTestKit
    with AnyWordSpecLike
    with Matchers
    with LogCapturing {

  "a sharded freight entity" should {
    "be able to add a cargo" in {

      val sharding = ClusterSharding(system)
      val entityDefinition =
        Entity(Container.TypeKey)(createBehavior = entityContext =>
          Container(entityContext.entityId)))
```